

Day 2: Pandas Foundations

Pandas is the workhorse of data work in Python. It gives you a table, the DataFrame, and hundreds of methods to select, filter, summarise, and reshape it. Today you learn the moves you will use every single day.

What you will learn

Core concepts

- The Series and the DataFrame, and how they relate
- Selecting rows and columns with labels and positions
- Filtering with boolean conditions
- Summary statistics and value counts
- Checking and handling missing data
- Reading and writing files

Technical tools

- `pandas` (`read_csv` , `loc` , `iloc` , `groupby` , `describe` , `to_csv`)
- A retail price optimization dataset

Dataset

We use a real **retail price optimization** dataset: 676 product-month records for 52 products across 9 categories. Each row holds a product's price, quantity sold, freight cost, product attributes (weight, rating, photo count), the count of weekdays/weekends/holidays in that month, and three competitors' prices for a similar product.

Additional reading

- Pandas *Getting started* tutorials: https://pandas.pydata.org/docs/getting_started/index.html
- *Python for Data Analysis* by Wes McKinney (pandas creator), chapters 5 to 6: <https://wesmckinney.com/book/>
- Pandas *10 minutes to pandas*: https://pandas.pydata.org/docs/user_guide/10min.html

Setup

```
In [ ]: import pandas as pd
import numpy as np
from pathlib import Path

def load_retail_price():
    candidates = ["W2_Data/retail_price.csv", "retail_price.csv", "../W2_Data/retail_price.csv"]
    for path in candidates:
        if Path(path).exists():
            return pd.read_csv(path)
    raise FileNotFoundError(
        "Could not find retail_price.csv. Adjust path to dataset."
    )

retail = load_retail_price()
print(retail.shape[0])

print("Shape:", retail.shape)
retail.head(10)
```

Shape: (676, 30)

Out[]:

	product_id	product_category_name	month_year	qty	total_price	freight_price	unit_price	product_name_length
0	bed1	bed_bath_table	01-05-2017	1	45.95	15.100000	45.950000	3
1	bed1	bed_bath_table	01-06-2017	3	137.85	12.933333	45.950000	3
2	bed1	bed_bath_table	01-07-2017	6	275.70	14.840000	45.950000	3
3	bed1	bed_bath_table	01-08-2017	4	183.80	14.287500	45.950000	3
4	bed1	bed_bath_table	01-09-2017	2	91.90	15.100000	45.950000	3
5	bed1	bed_bath_table	01-10-2017	3	137.85	15.100000	45.950000	3
6	bed1	bed_bath_table	01-11-2017	11	445.85	15.832727	40.531818	3
7	bed1	bed_bath_table	01-12-2017	6	239.94	15.230000	39.990000	3
8	bed1	bed_bath_table	01-01-2018	19	759.81	16.533684	39.990000	3
9	bed1	bed_bath_table	01-02-2018	18	719.82	13.749444	39.990000	3

10 rows × 30 columns

◀			▶
---	--	--	---

1. Series and DataFrame

A **Series** is a single column: values plus an index (the row labels). A **DataFrame** is a set of Series sharing one index, so it is a full table.

Pulling one column out of a DataFrame gives you a Series.

```
In [ ]: price_col = retail["unit_price"]
print(type(price_col))
price_col.head()
```

<class 'pandas.core.series.Series'>

Out[]:

	unit_price
0	45.95
1	45.95
2	45.95
3	45.95
4	45.95

dtype: float64

2. First look at any dataset

Before analysis, always inspect. These four calls answer "what am I holding?"

- `.shape` : rows and columns
- `.head()` / `.tail()` : a few rows
- `.info()` : column names, types, and non-null counts
- `.describe()` : summary statistics for numeric columns

```
In [ ]: retail.tail()
```

Out[]:

	product_id	product_category_name	month_year	qty	total_price	freight_price	unit_price	product_name_length
671	bed5	bed_bath_table	01-05-2017	1	215.00	8.760000	215.000000	
672	bed5	bed_bath_table	01-06-2017	10	2090.00	21.322000	209.000000	
673	bed5	bed_bath_table	01-07-2017	59	12095.00	22.195932	205.000000	
674	bed5	bed_bath_table	01-08-2017	52	10375.00	19.412885	199.509804	
675	bed5	bed_bath_table	01-09-2017	32	5222.36	24.324687	163.398710	

5 rows × 30 columns

◀			▶
---	--	--	---

--

```
In [ ]: retail.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 676 entries, 0 to 675
Data columns (total 30 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   product_id                            676 non-null    object
1   product_category_name                 676 non-null    object
2   month_year                           676 non-null    object
3   qty                                   676 non-null    int64
4   total_price                           676 non-null    float64
5   freight_price                         676 non-null    float64
6   unit_price                           676 non-null    float64
7   product_name_lenght                  676 non-null    int64
8   product_description_lenght           676 non-null    int64
9   product_photos_qty                   676 non-null    int64
10  product_weight_g                     676 non-null    int64
11  product_score                         676 non-null    float64
12  customers                             676 non-null    int64
13  weekday                              676 non-null    int64
14  weekend                               676 non-null    int64
15  holiday                              676 non-null    int64
16  month                                676 non-null    int64
17  year                                 676 non-null    int64
18  s                                    676 non-null    float64
19  volume                              676 non-null    int64
20  comp_1                              676 non-null    float64
21  ps1                                  676 non-null    float64
22  fp1                                  676 non-null    float64
23  comp_2                              676 non-null    float64
24  ps2                                  676 non-null    float64
25  fp2                                  676 non-null    float64
26  comp_3                              676 non-null    float64
27  ps3                                  676 non-null    float64
28  fp3                                  676 non-null    float64
29  lag_price                            676 non-null    float64
dtypes: float64(15), int64(12), object(3)
memory usage: 158.6+ KB
```

```
In [ ]: retail.describe()
```

Out[]:

	qty	total_price	freight_price	unit_price	product_name_lenght	product_description_lenght	prod
count	676.000000	676.000000	676.000000	676.000000	676.000000	676.000000	
mean	14.495562	1422.708728	20.682270	106.496800	48.720414	767.399408	
std	15.443421	1700.123100	10.081817	76.182972	9.420715	655.205015	
min	1.000000	19.900000	0.000000	19.900000	29.000000	100.000000	
25%	4.000000	333.700000	14.761912	53.900000	40.000000	339.000000	
50%	10.000000	807.890000	17.518472	89.900000	51.000000	501.000000	
75%	18.000000	1887.322500	22.713558	129.990000	57.000000	903.000000	
max	122.000000	12095.000000	79.760000	364.000000	60.000000	3006.000000	

8 rows × 27 columns



```
In [ ]: retail[["qty", "unit_price", "freight_price", "customers", "product_score"]].describe()
```

Out[]:

	qty	unit_price	freight_price	customers	product_score
count	676.000000	676.000000	676.000000	676.000000	676.000000
mean	14.495562	106.496800	20.682270	81.028107	4.085503
std	15.443421	76.182972	10.081817	62.055560	0.232021
min	1.000000	19.900000	0.000000	1.000000	3.300000
25%	4.000000	53.900000	14.761912	34.000000	3.900000
50%	10.000000	89.900000	17.518472	62.000000	4.100000
75%	18.000000	129.990000	22.713558	116.000000	4.200000
max	122.000000	364.000000	79.760000	339.000000	4.500000

.info() lists 30 columns: identifiers (product_id , product_category_name , month_year), the target we care

about (`qty`), pricing fields (`unit_price` , `freight_price` , competitor prices), and product and calendar attributes. We look at whether any of them have gaps in section 6.

3. Selecting columns and rows

Two accessors do most selection:

- `.loc[rows, cols]` selects by **label**.
- `.iloc[rows, cols]` selects by **integer position**.

Pick columns by passing a list of names.

```
In [ ]: # A few columns, all rows
retail[["product_id", "product_category_name", "unit_price", "qty"]].head()
```

```
Out[ ]:   product_id  product_category_name  unit_price  qty
0         bed1         bed_bath_table        45.95    1
1         bed1         bed_bath_table        45.95    3
2         bed1         bed_bath_table        45.95    6
3         bed1         bed_bath_table        45.95    4
4         bed1         bed_bath_table        45.95    2
```

```
In [ ]: # .loc by label: rows 0 to 3, two named columns
print(retail.loc[0:4, ["product_id", "month_year"]])

print()

# .iloc by position: first 3 rows, first 2 columns
print(retail.iloc[0:5, 0:2])
```

```
product_id  month_year
0         bed1  01-05-2017
1         bed1  01-06-2017
2         bed1  01-07-2017
3         bed1  01-08-2017
4         bed1  01-09-2017
```

```
product_id  product_category_name
0         bed1         bed_bath_table
1         bed1         bed_bath_table
2         bed1         bed_bath_table
3         bed1         bed_bath_table
4         bed1         bed_bath_table
```

```
In [ ]: retail.loc[0]
```

Out[]:

0

product_id	bed1
product_category_name	bed_bath_table
month_year	01-05-2017
qty	1
total_price	45.95
freight_price	15.1
unit_price	45.95
product_name_lenght	39
product_description_lenght	161
product_photos_qty	2
product_weight_g	350
product_score	4.0
customers	57
weekday	23
weekend	8
holiday	1
month	5
year	2017
s	10.267394
volume	3800
comp_1	89.9
ps1	3.9
fp1	15.011897
comp_2	215.0
ps2	4.4
fp2	8.76
comp_3	45.95
ps3	4.0
fp3	15.1
lag_price	45.9

dtype: object

4. Filtering with conditions

A comparison on a column returns a boolean Series. Passing it back into the DataFrame keeps only rows where the condition is True. This is the same boolean masking idea from Day 1, now on a table.

Combine conditions with `&` (and) and `|` (or). Wrap each condition in parentheses.

In []:

```
# health beauty products that sold more than 15 units in a month
print("Shape", retail.shape)
mask = (retail["product_category_name"] == "health_beauty") | (retail["qty"] > 15)
strong_month = retail[mask]
print("Rows found:", len(strong_month))
strong_month[["product_id", "month_year", "unit_price", "qty"]].head()
```

Shape (676, 30)

Rows found: 295

```
Out[ ]:
```

	product_id	month_year	unit_price	qty
8	bed1	01-01-2018	39.990000	19
9	bed1	01-02-2018	39.990000	18
10	bed1	01-03-2018	39.990000	17
12	bed1	01-05-2018	39.990000	19
18	garden5	01-05-2017	97.588235	20

```
In [ ]: mask
```

```
Out[ ]:
```

	0
0	False
1	False
2	False
3	False
4	False
...	...
671	False
672	False
673	True
674	True
675	True

676 rows × 1 columns

dtype: bool

5. Summaries and grouping

`.value_counts()` counts how often each category appears. `.groupby()` splits the table by a category, then applies a summary to each group. This split, apply, combine pattern is one of the most useful tools in pandas.

```
In [ ]: print(retail["product_category_name"].unique())
```

['bed_bath_table' 'garden_tools' 'consoles_games' 'health_beauty'
'cool_stuff' 'perfumery' 'computers_accessories' 'watches_gifts'
'furniture_decor']

```
In [ ]: print(retail["product_category_name"].value_counts())
```

```
product_category_name
garden_tools          160
health_beauty         130
watches_gifts         103
computers_accessories   69
bed_bath_table         61
cool_stuff            57
furniture_decor        48
perfumery             26
consoles_games        22
Name: count, dtype: int64
```

```
In [ ]: # Mean unit price per category
average_price_product_category = retail.groupby("product_category_name")["unit_price"].mean().round(2).sort_valu
average_price_product_category
```

Out[]: **unit_price**

product_category_name	
watches_gifts	164.88
health_beauty	132.31
computers_accessories	119.48
cool_stuff	107.86
perfumery	89.35
garden_tools	80.09
bed_bath_table	78.63
furniture_decor	60.15
consoles_games	27.03

dtype: float64

```
In [ ]: average_price_product_category.rank(ascending=False)
```

Out[]: **unit_price**

product_category_name	
watches_gifts	1.0
health_beauty	2.0
computers_accessories	3.0
cool_stuff	4.0
perfumery	5.0
garden_tools	6.0
bed_bath_table	7.0
furniture_decor	8.0
consoles_games	9.0

dtype: float64

```
In [ ]: average_price_product_category.loc["watches_gifts"]
average_price_product_category['rank'] =
average_price_product_category
```

```

-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_1250/456641632.py in <cell line: 0>()
      1 average_price_product_category.loc["watches_gifts"]
----> 2 average_price_product_category['rank'] = average_price_product_category.rank()
      3 average_price_product_category

/usr/local/lib/python3.13/dist-packages/pandas/core/generic.py in rank(self, axis, method, numeric_only, na_opti
on, ascending, pct)
    10102         data = self
    10103
> 10104         return ranker(data)
    10105
    10106         @doc(_shared_docs["compare"], klass=_shared_doc_kwargs["klass"])

/usr/local/lib/python3.13/dist-packages/pandas/core/generic.py in ranker(data)
    10079         )
    10080         else:
> 10081             ranks = algos.rank(
    10082                 values,
    10083                 axis=axis_int,

/usr/local/lib/python3.13/dist-packages/pandas/core/algorithms.py in rank(values, axis, method, na_option, ascen
ding, pct)
    1100
    1101     if values.ndim == 1:
-> 1102         ranks = algos.rank_1d(
    1103             values,
    1104             is_datetimelike=is_datetimelike,

algos.pyx in pandas._libs.algos.rank_1d()

/usr/local/lib/python3.13/dist-packages/pandas/core/generic.py in __nonzero__(self)
    1575     @final
    1576     def __nonzero__(self) -> NoReturn:
-> 1577         raise ValueError(
    1578             f"The truth value of a {type(self).__name__} is ambiguous. "
    1579             "Use a.empty, a.bool(), a.item(), a.any() or a.all()."

ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(), a.item(), a.any() or a.all().

```

```

In [ ]: # Several statistics per category at once
        retail.groupby("product_category_name")["qty"].agg(["count", "mean", "min", "max"]).round(1)

```

```

Out[ ]:

```

	count	mean	min	max
product_category_name				
bed_bath_table	61	16.8	1	60
computers_accessories	69	16.9	1	71
consoles_games	22	10.9	1	28
cool_stuff	57	9.8	1	76
furniture_decor	48	18.6	1	122
garden_tools	160	15.0	1	91
health_beauty	130	14.2	1	58
perfumery	26	9.4	1	24
watches_gifts	103	13.9	1	114

6. Checking and handling missing data

Real data can have holes, and pandas marks them as `NaN`. The first step is always to check, whatever the source. This dataset happens to be a curated export with no gaps, which is worth confirming rather than assuming.

```

In [ ]: retail.info()

```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 676 entries, 0 to 675
Data columns (total 30 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   product_id                            676 non-null    object
1   product_category_name                 676 non-null    object
2   month_year                           676 non-null    object
3   qty                                    676 non-null    int64
4   total_price                           676 non-null    float64
5   freight_price                         676 non-null    float64
6   unit_price                           676 non-null    float64
7   product_name_lenght                  676 non-null    int64
8   product_description_lenght           676 non-null    int64
9   product_photos_qty                   676 non-null    int64
10  product_weight_g                      676 non-null    int64
11  product_score                         676 non-null    float64
12  customers                             676 non-null    int64
13  weekday                               676 non-null    int64
14  weekend                               676 non-null    int64
15  holiday                               676 non-null    int64
16  month                                 676 non-null    int64
17  year                                 676 non-null    int64
18  s                                     676 non-null    float64
19  volume                               676 non-null    int64
20  comp_1                               676 non-null    float64
21  ps1                                   676 non-null    float64
22  fp1                                   676 non-null    float64
23  comp_2                               676 non-null    float64
24  ps2                                   676 non-null    float64
25  fp2                                   676 non-null    float64
26  comp_3                               676 non-null    float64
27  ps3                                   676 non-null    float64
28  fp3                                   676 non-null    float64
29  lag_price                             676 non-null    float64
dtypes: float64(15), int64(12), object(3)
memory usage: 158.6+ KB
```

```
In [ ]: # Count missing values per column
        retail.isna().sum().sum()
```

```
Out[ ]: np.int64(0)
```

Zero missing values across all 30 columns. That is not always the case, so the habit still matters: `.isna().sum()` per column, then decide. Since this dataset came already aggregated to the product-month level by whoever built it, most of the cleaning work happened before it reached us, but you should never take that on faith.

To keep the two repair strategies fresh, here is how you would apply them if a column did have gaps, using `freight_price` as the example column:

```
In [ ]: # Option A: drop rows with a missing value in a given column
        would_drop = retail.dropna(subset=["freight_price"])
        print("Rows if we dropped missing freight_price:", would_drop.shape[0], "(unchanged here, no gaps exist)")

        # Option B: fill missing values in a column with its median
        filled_example = retail.copy()
        filled_example["freight_price"] = filled_example["freight_price"].fillna(filled_example["freight_price"].mean())
        print("Missing freight_price after fill:", filled_example["freight_price"].isna().sum())
```

```
Rows if we dropped missing freight_price: 676 (unchanged here, no gaps exist)
Missing freight_price after fill: 0
```

Dropping is simple but loses rows; filling keeps rows but invents values. The right choice depends on how much is missing and why. We carry `retail` forward unchanged for the rest of the week, since it is already complete. Generally, a 5% threshold rule is followed for handling missing data:

1. Less than 5% missing: We can safely drop the rows with missing values, as this small loss will not significantly impact the statistical power of the dataset.
2. More than 5% missing: We avoid dropping data to prevent losing valuable information. Instead, we fill (impute) the missing values based on the column's data type: Numerical columns: Filled using the mean (for normally distributed data) or median (if skewed or containing outliers).
Categorical columns: Filled using the mode (the most frequent value).

Exercise (10 to 15 minutes)

Use the `retail` DataFrame. Work top to bottom. Each task uses one idea from today.

```
In [ ]: # Work with the full dataset
```

```
df = retail.copy()

# Task 1: how many rows and columns does df have? Print the shape.

# Task 2: select only the columns product_category_name, unit_price, and qty.

# Task 3: filter to rows in the category "garden_tools" with unit_price
#          below 50. How many are there?

# Task 4: using groupby, find the mean qty for each product_category_name.

# Task 5: use value_counts to show how many product-months fall in each year.

# Task 6 (stretch): write your filtered result from Task 3 to
#                  "cheap_garden_tools.csv" without the index column.
```

Solution

```
In [ ]: # Task 1
print("Shape:", df.shape)

# Task 2
print(df[["product_category_name", "unit_price", "qty"]].head())

# Task 3
cheap_garden = df[(df["product_category_name"] == "garden_tools") & (df["unit_price"] < 50)]
print("Cheap garden tools rows:", len(cheap_garden))

# Task 4
print(df.groupby("product_category_name")["qty"].mean().round(1))

# Task 5
print(df["year"].value_counts())

# Task 6
cheap_garden.to_csv("cheap_garden_tools.csv", index=False)
print("Saved cheap_garden_tools.csv")
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[16], line 2
      1 # Task 1
----> 2 print("Shape:", df.shape)
      3
      4 # Task 2
      5 print(df[["product_category_name", "unit_price", "qty"]].head())

NameError: name 'df' is not defined
```

Recap

- A Series is one column, a DataFrame is a table of aligned Series.
- Inspect first: `shape`, `head`, `info`, `describe`.
- Select with `.loc` (labels) and `.iloc` (positions); filter with boolean conditions joined by `&` and `|`.
- `groupby` and `value_counts` summarise categories.
- Always count missing values before assuming there are none; decide per column whether to drop or fill.
- `read_csv` and `to_csv` move data in and out; use `index=False` when writing.

Tomorrow we reshape and pivot this same pricing data into the exact form an analysis needs.